

SYSTEM DESIGN

Principles -

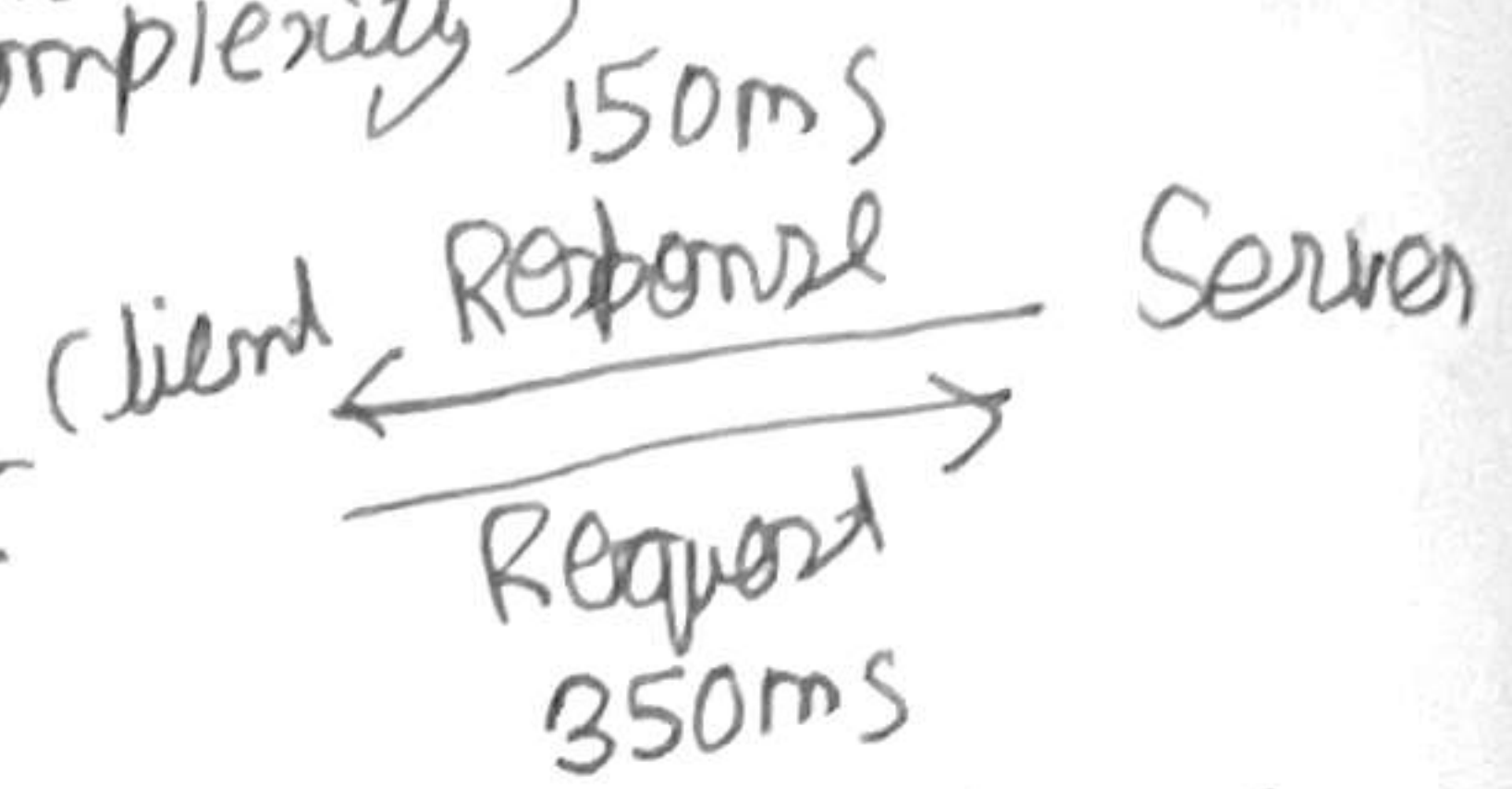
- 1) Scalability = to extend the computer network on large level
- 2) Availability = system/computer is active (online/offline)
- 3) Reliability = how well the system can tolerate the fault in system

Monolithic & Microservices (architecture) -

- (i) Monolithic = subset under which one user like CIMS (all details under one side of students)
- (ii) Microservices = small & independent systems (increased operational complexity)

Latency & throughput (Performance metrics) -

- (i) Latency = how much time data packets takes to react
- (ii) Throughput = measures the volume of data points



$$\text{Latency} = \text{Request} + \text{Response} = 500\text{ms}$$

Cache = Improves the efficiency of RAM

SQL vs NoSQL -

- (i) SQL = Relational database & ; Defined structured schema ; ACID principle
- (ii) NoSQL = No defined structure - pre-formed database ; BASE principle

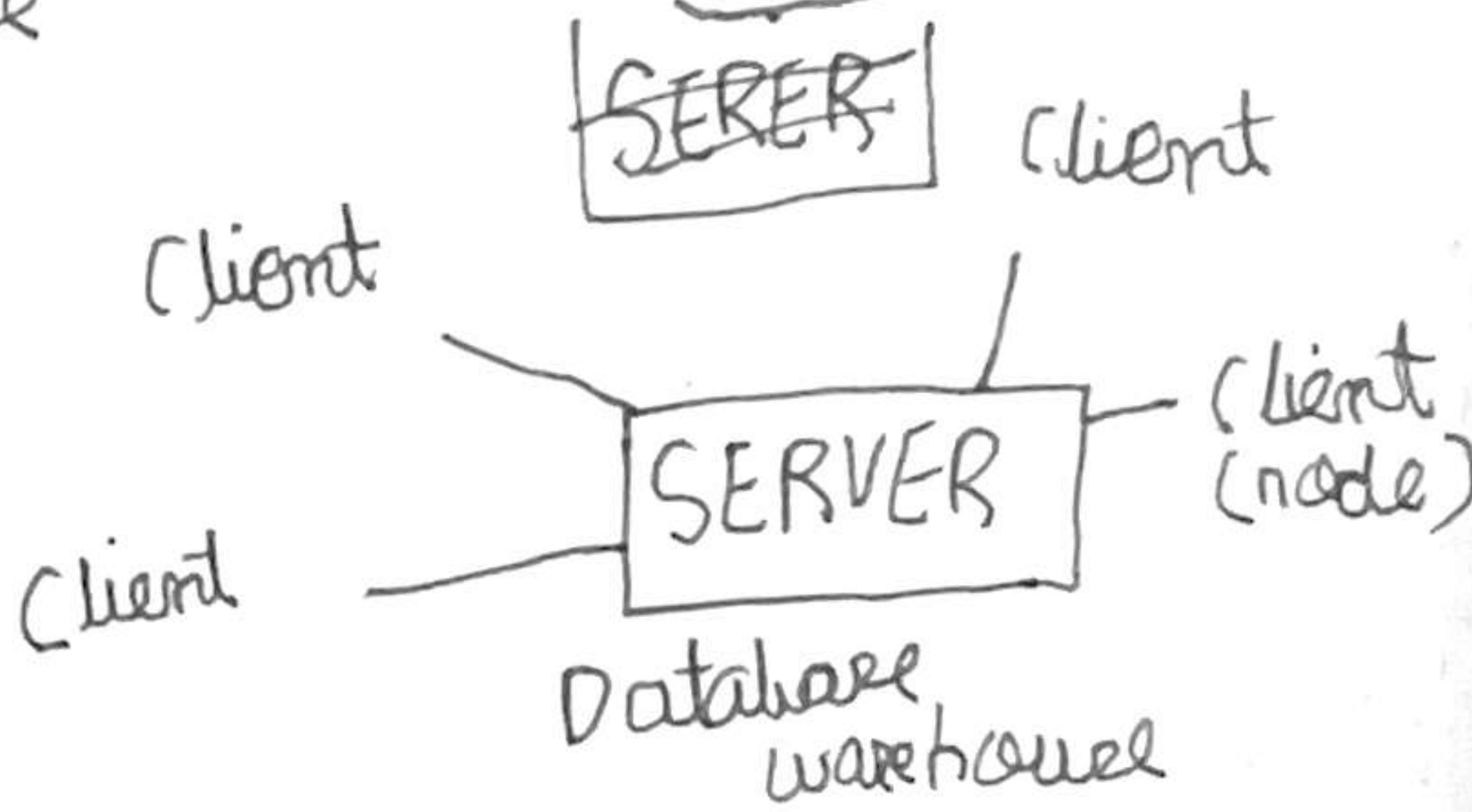
BASE principle
 Basically Available
 Soft state
 Eventual consistency

Load Balancing = to distribute load of database

Redundant = duplicate/repetitive data

Distributed database (Parallel attribute) -

Primary database <--> to many systems



Fragmentation = data in small fragments

Replication = copying

Concurrency control = handle multiple transaction

Caching - (Synchronization is challenge)



eg- Call Logs, passwords

Introduction to system design -

Steps -

- 1) Planning
- 2) Feasibility study
- 3) System design
- 4) Implementation
- 5) Testing
- 6) Deployment
- 7) Maintenance & support

yields outputs

Functional requirements = Defines specific behaviors & functions the system must support
(The "What")

eg - User can send a message
System generates an invoice
Admin can liaise users

Non-Functional requirements = Defines the quality attributes of the system
(The "How, Much")

eg - Scalability: Handle 10k req/sec
Latency: < 200ms response
Availability: 99.99% uptime

Scaling Strategies - Easier management

(i) Vertical scaling: adding more power to existing server.
(Scale up) Simple but has a hard ceiling (hardware limit)

(ii) Horizontal scaling: adding more servers to your resource pool.
(Scale out) requires complex to distribute load but offers infinite theoretical limit. Improved performance, fault tolerance

Database Sharding - Involves splitting a large database into smaller, faster, more easily managed parts called shards

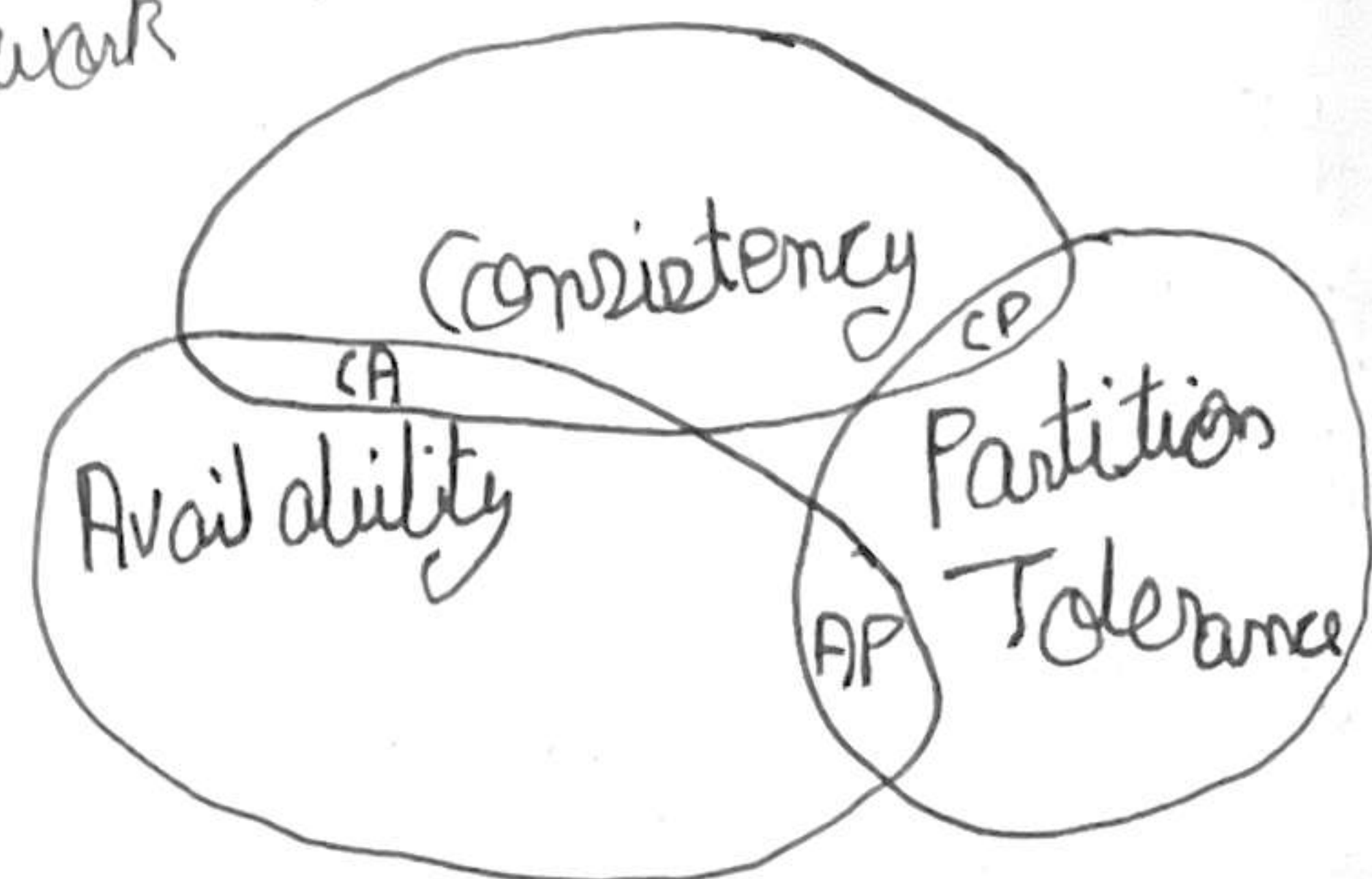
Horizontal Partitioning = Rows are distributed across different servers based on a Shard Key
Pros: Unlimited storage scaling
Cons: Complex queries (joins) across shards are difficult

CAP Theorem: In a distributed data store, you can only guarantee 2 of the following three:

(i) Consistency = every read receives the most recent write
(ii) Availability = every request receives a response
(iii) Partition Tolerance = system works despite network failures

Limitations of scalability -

(i) Cost vs scalability
(ii) Latency vs Throughput
(iii) Complexity
(iv) Data consistency



SQL vs Non SQL (5M)

Aspect 1) Schema

- SQL Databases
- Fixed, predefined schema
 - Changes require migrations

- Non SQL Databases
- Dynamic, flexible schema
 - Easy to modify structure

2) Query Language

- Standardized SQL across platforms

- Database specific query languages

3) Transaction

- Multi row ACID transactions

- Limited transaction support (BASE principle)

4) Data Structure

- Structured, relational data

- Unstructured, semi structured

5) Use Cases

- Financial systems
ERP, complex queries requiring joins

- Real time analytics
- Content management
- Social networks

SQL = Relational

Non SQL = Non Relational

#CAP Trade-Offs -

~~In distributed systems, network partitions are inevitable - cables break, routers~~

(i) CA Systems (Consistency + Availability):

- Prioritize consistency & availability but cannot tolerate partitions
- These systems work well in single node or tightly coupled environments
- eg. Traditional RDBMS in a single data centre

(ii) CP Systems (Consistency + Partition Tolerance):

- Maintain consistency even during network partitions, sacrificing availability.
- Some nodes may become unavailable to prevent inconsistent data
- eg. MongoDB, HBase, Financial transactions

(iii) AP Systems (Availability + Partition Tolerance)

- Prioritize availability during partitions, accepting eventual consistency.
- All nodes remain accessible but may serve stale data
- eg. Cassandra, DynamoDB, social media feeds

Replication -

- Systems store copies of data on different sites
- If an entire database is available on multiple sites, it is a fully redundant database
- Advantage: it increases data availability on different sites and allows for parallel query requests to be processed

Fragmentation -

- When it comes to fragmentation of distributed database storage, the relations are fragmented, which means they are split into smaller parts.
- Each of the fragments are stored on a different, where it is required.
- Prerequisite for fragmentation is to make sure that the fragments can later be reconstructed into the original relation without losing data
- Types - [Internal
External

Issues of Distributed databases -

- (i) Costly software
- (ii) Large overhead (requires numerous calculations & constant synchronization)
- (iii) Data Integrity

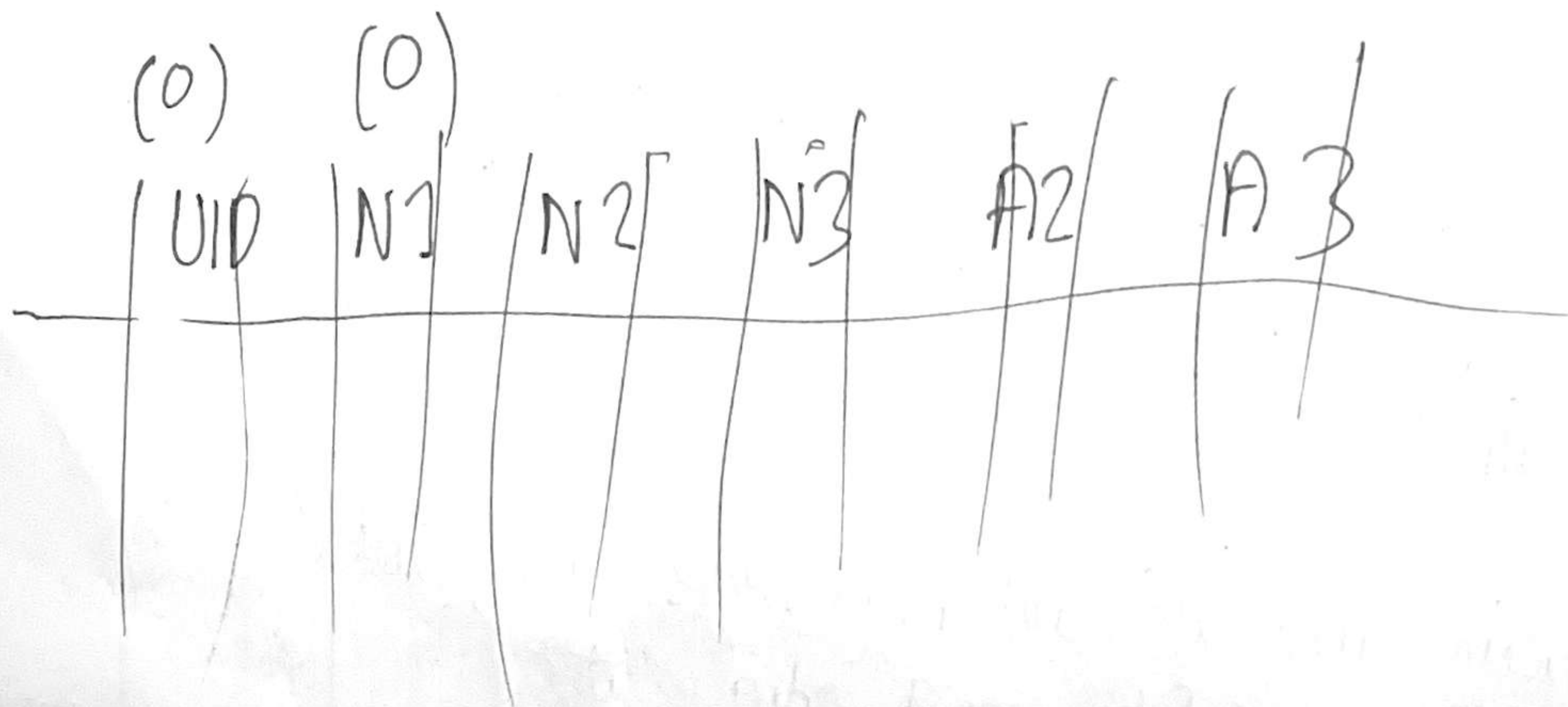
Centralized vs Distributed Database -

Centralized Database

- (i) Database are stored in only one site
- (ii) Database is maintained at one site
- (iii) If centralized system fails, entire system is halted
- (iv) It is less reliable

Distributed Database

- (i) Database are stored in different site & help of network it can access it
- (ii) Database is maintained at a number of different sites
- (iii) If one system fails, system continues work with other site
- (iv) It is more reliable



ASSIGNMENT-1

1. Differentiate between functional & non functional requirements with an example.
2. Compare horizontal & vertical scaling. Explain when each is suitable
3. What is load balancing & describe any 2 load balancing algorithms.
4. Differentiate between client side & server side ~~and~~ caching with their advantages.
5. What is database sharding? Explain different sharding techniques.
6. Explain replication in ~~sharding~~ systems. Describe various replication techniques. distributed

Deadline = 3 Feb 2026

<< DO ANY 4! >>

Distributed Database -

- A database that runs and stores data across multiple computers, as opposed to doing everything on a single machine.
- Typically, they operate on 2 or more interconnected servers on a computer network
- Each location where a version of the database is running is called an instance or node
- A distributed database is basically a database that is not limited to one system, it is spread over different sites i.e. on multiple computers ~~over~~ or over a network of computers
- A ~~diff~~ distributed database system is located on ~~a~~ various sites that don't share physical components
- Applications:
 - └ Used in corporate management information system
 - └ Used in multimedia applications
 - └ Used in military's control system, hotel chains, etc
 - └ Used in manufacturing control system

- Types:

- └ Homogeneous
- └ Heterogeneous
- Managed in 2 ways:
 - └ Replication
 - └ Fragmentation



(i) Homogeneous Database:

- All diff sites store database identically
- The OS, DBMS and data structure used all are the same at all sites. Hence, they're easy to manage.

(ii) Heterogeneous Database:

- Allows nodes in a distributed system to use different DBMS software or possess varying database schemas.
- This approach caters to diverse requirements and facilitates seamless integration between nodes employing different technologies.

Microservices architectures -

- Extends to the loosely coupled services which can be developed, deployed and maintained independently.
- Each of these services is responsible for discrete task and can communicate with other services through simple APIs to solve a larger complex business problem.

- Benefits:

- Services are small and modular
- Clear service boundaries make development & scaling easier
- Services can be deployed independently
- Easy scalability
- Fault isolation
- Faster updates

Monolithic Architectures -

- Contains all application components into a single codebase
- Preferred for its simplicity and ease of initial setup
- Divide the application into smaller, separately deployable services
- It is difficult to scale and maintain, which makes it difficult to adjust to changing needs

- Characteristics:

- Single codebase
- Tight coupling (closely linked)
- Shared memory
- Centralized database
- Layered structure

- Importance:

- Simplicity
- Cost effectiveness
- Performance
- Security
- Legacy support

- Challenges:

- Long deployment cycles
- Risk of downtime

Q. How to design the individual services?

- Ans
- (i) Clearly define service boundaries & responsibilities
 - (ii) Decide what to expose & how services communicate
 - (iii) Hide internal logic & implementation details
 - (iv) Expose only required interfaces
 - (v) Avoid tight coupling for better flexibility
-

Q. How can a system grow?

- Ans
- (i) More users
 - (ii) More features
 - (iii) More data
 - (iv) More complexity
 - (v) More geographies
-

Growth in user base -

More users started using the system, leading to increased no. of requests

Growth in features -

More features were introduced to expand the system's capabilities

Growth in data volume -

Growth in the amount of data, the system stores & manages due to user activity or logging

Growth in complexity -

The system's architecture evolves to accommodate new features, scale or integrations, resulting in additional components & dependencies

Growth in Geographic reach -

System is expanded to serve users in new regions or countries

Vertical Scaling -

Advantages

- └ Increased capacity
- └ Easier management

Disadvantages

- └ Limited scalability

Horizontal Scaling -

Advantages

- └ Increased capacity
- └ Improved performance
- └ Increased fault tolerance

Disadvantages

- └ Complex system design
- └ High cost

Applications of Scalable systems -

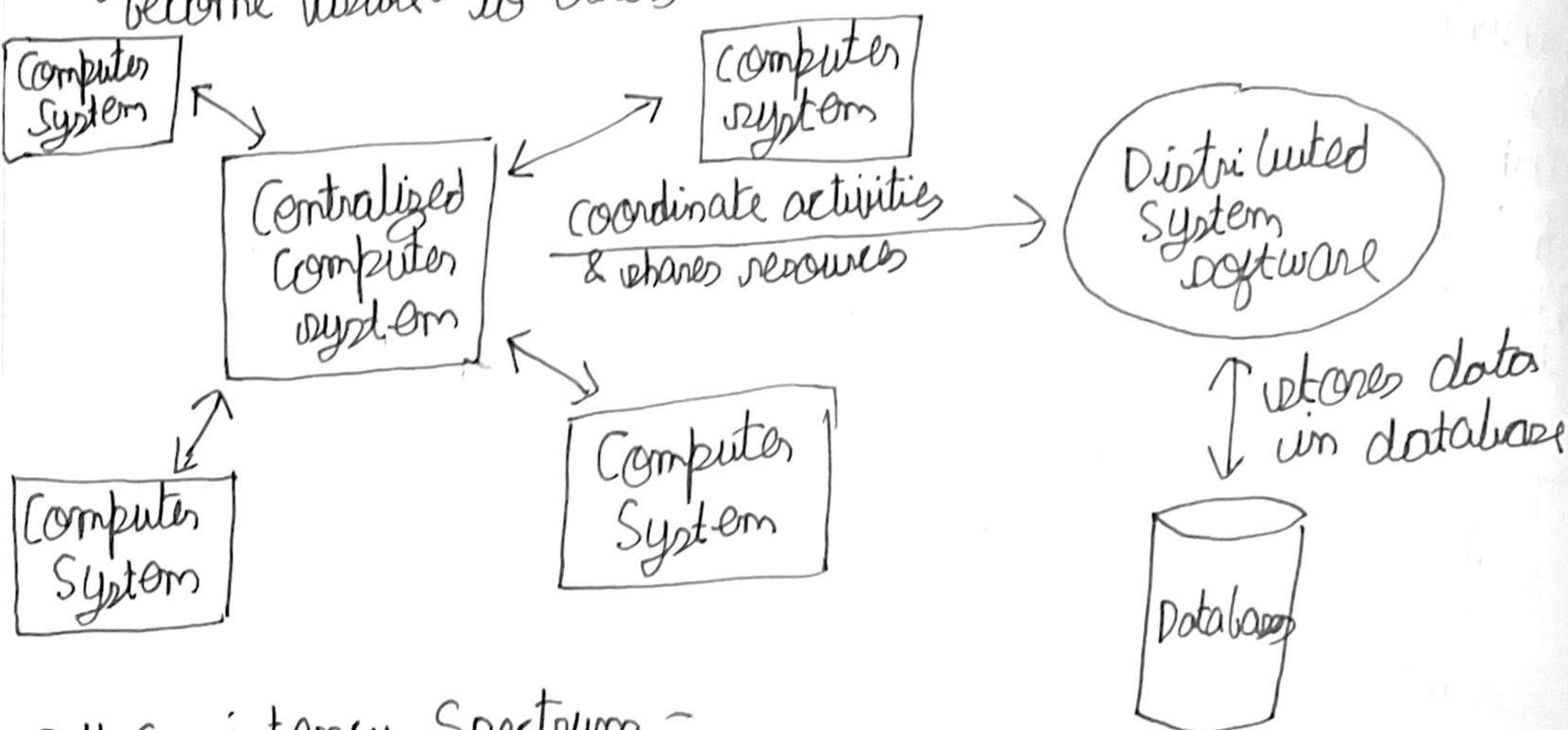
- Google
- AWS
- Netflix

Challenges & Tradeoffs in Scalable systems -

- (i) Cost vs scalability
- (ii) Complexity
- (iii) Latency & throughput
- (iv) Data partitioning tradeoffs

Consistency Models -

- Acts as a contractual agreement between a distributed data store and the processes that access it
- Defines the rules for how & when updates made by one node become visible to others



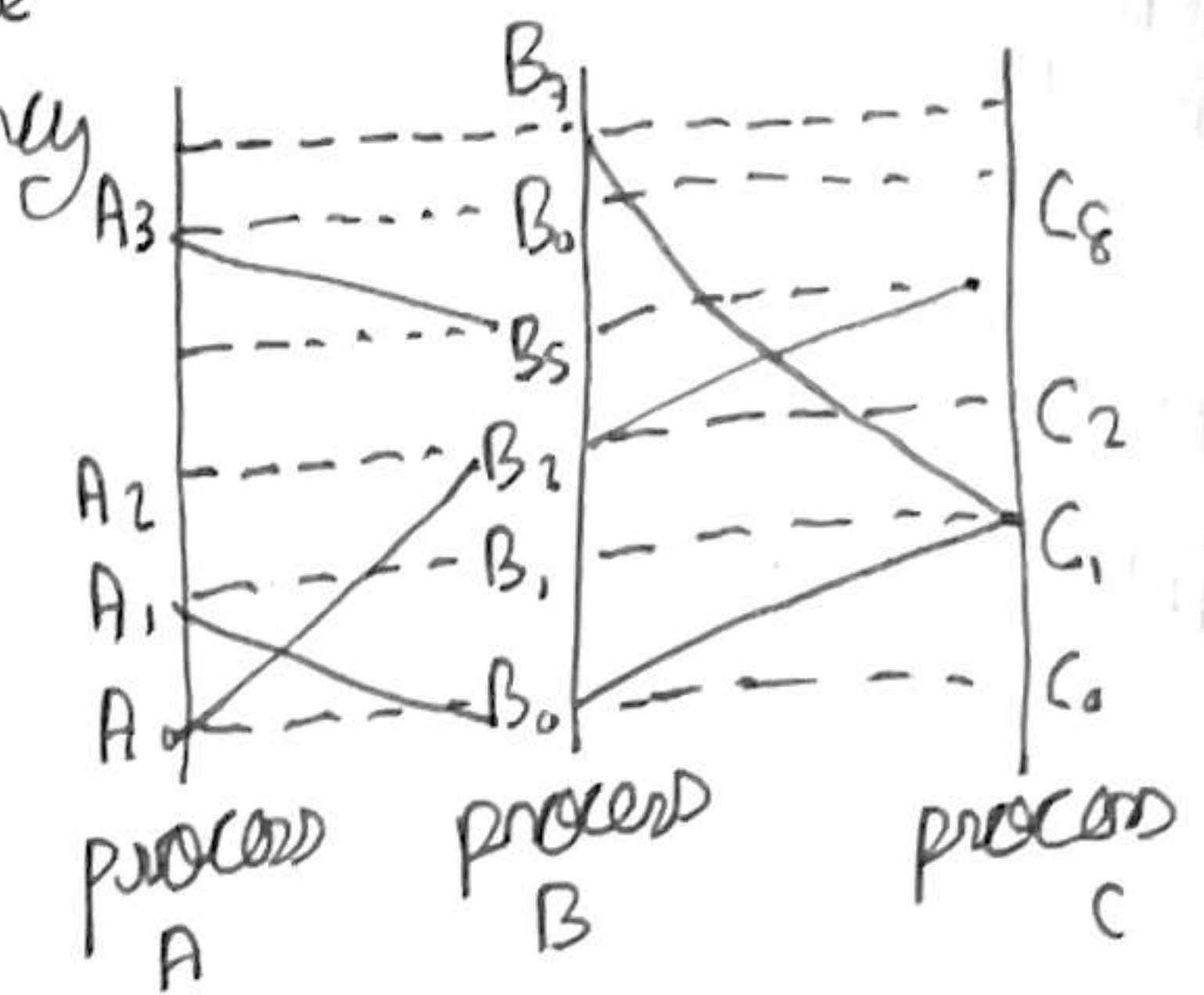
Consistency Spectrum -

- (i) Strong Linearizability
 - ↳ System behaves as a single copy. Highest latency
- (ii) Sequential ordering preserved
 - ↳ Operations appear in a specific order, but not necessarily real time
- (iii) ~~Causal~~ Causal cause & effect
 - ↳ related events are seen in order; unrelated events can be parallel
- (iv) Eventual convergence
 - ↳ replicas converge eventually.
 - Lowest latency, high availability

Linearizability —

(Strict consistency)

- Strongest consistency model
- Operations appear to happen instantaneously
- Real time (wall clock) order is preserved
- Once a write completes, all later reads return that value
- System behaves like a single machine
- Easy to reason, but causes high latency



Sequential Consistency —

- Weaker than linearizability
- Does not require real time order
- All operations appear in some sequential order

- Order of operations within the same process is preserved

- Different processes may see operations in different time orders
- Useful where order matters more than exact time

Linearizability vs Sequential Consistency —

Aspect	Linearizability	Sequential Consistency
1) Real time order	Required	Not required
2) Strength	Stronger	Weaker
3) System illusion	Single machine	Logical sequence
4) Latency	Higher	Lower
5) Easy to implement	No	Yes

- ## # Eventual consistency -
- Weak consistency model used in distributed systems
 - Data updates happen at different replicas asynchronously
 - Temporary inconsistency is allowed. All replicas become consistent eventually, after updates propagate
 - Commonly used when availability and performance matter more than immediate accuracy

Why Eventual Consistency is

- High availability
- Low Latency
- Scalability

- ## # Working :
- A client writes data to a single replica or leader node
 - The update is added to a replication log
 - Replicas receive the update asynchronously
 - All replicas converge to the same state over time

Strong vs Eventual Consistency -

Aspect	Strong consistency	Eventual consistency
1) Data Sync	Immediate	Delayed
2) Read behavior	Always latest data	Might see stale data
3) Availability	Lower	Very high
4) Latency	Higher	Very low
5) Use Cases	Banking, payments	Social media, messaging
6) Data sync	Immediate	Delayed

Advantages:

- High accountability
- Fault tolerant
- High performance
- Supports geo distributed users

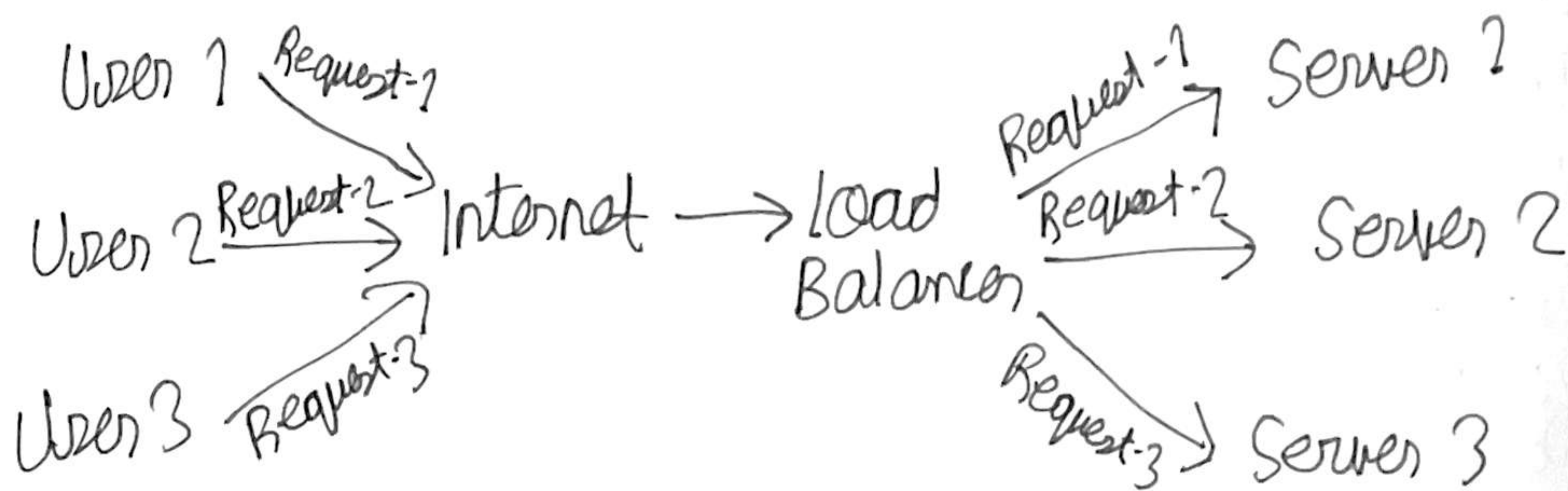
Challenges:

- Stale reads
- Conflict resolution
- Difficult to reason about

~~Eventual Consistency~~

Load Balancing -

- Sits between clients & servers
- Acts as a traffic cop, distributing ^{incoming} ~~incoming~~ requests across multiple healthy servers
- Prevents overload on a single server
- Increases availability
- Algorithms: Round Robin, Least Connections, IP Hash



- Need:

- Performance improvement
- Better availability
- Overload prevention

- Benefits:

- Scalability
- Fault tolerance
- High throughput

Types of Load Balancers -

(i) Hardware Load Balancers

- Physical devices
- Very fast & reliable
- Expensive
- eg - F5, Citrix

(ii) Software Load Balancers

- Run on general purpose machines
- eg - NGINX, HAProxy

(iii) Cloud Load Balancers

- Managed by cloud providers
- Highly Scalable
- No maintenance required
- eg - AWS ELB/ALB, Google Cloud LB, Azure LB

Load Balancing Algorithms -

- Goal is to make sure that no single server is overloaded when too many users visit an application

- Types:

(i) Static

- Round Robin
- Weighted Round Robin
- Source IP hash

(ii) Dynamic

- Least connection method
- Least response time method

- Challenges:

- Session Persistence (users sometimes need to stay connected to a specific server)
- Global Traffic distribution
- Failover management

Static Load Balancing -

- Tasks are assigned beforehand
- No runtime adaptation
- Based on fixed allocation
- Simple but not suitable for dynamic workloads

1) Round Robin Load Balancing:

- Requests are assigned sequentially
- Each server gets equal turn
- Easy to implement
- Does not consider current load
- Can cause server overload

2) Weighted Round Robin Load Balancing:

- Similar to Round Robin
- Each server has a weight
- Higher weight $\rightarrow$ more requests
- Suitable for heterogeneous servers
- Still static, no real time feedback

3) Source IP Hash Load Balancing:

- Uses hash of client IP
- Same IP $\rightarrow$ same server
- Ensures session consistency
- Good for stateful applications
- Uneven load if IPs are skewed